

Calibrating Self-Exciting Processes from Counts

**The Mean Behavior Poisson Process:
theory, computation, and code**

A textbook companion to the `hawkes_calibration` package

Contents

Preface	v
I Foundations	1
1 Events you cannot see one by one	2
1.1 Three problems, one shape	2
1.2 The model we want, and why it resists	2
1.3 The plan: fit a well-behaved twin	3
1.4 What this book builds	3
2 Poisson processes, intensity, and compensators	5
2.1 Counting processes and conditional intensity	5
2.2 The compensator	5
2.3 The Poisson process and the theorem that runs the book	6
2.4 The likelihood (why it needs timestamps)	6
3 The Hawkes process and its cluster representation	7
3.1 Definition	7
3.2 The cluster (branching) representation	7
3.3 Why it is not Poisson, and the mean-intensity equation	8
II The Mean Behavior Poisson Process	10
4 The Mean Behavior Poisson process	11
4.1 Definition and parameter correspondence	11
4.2 Why it is Poisson — and why that is the point	11
4.3 In code	12
5 Solving the MBPP equation	13
5.1 A feedback loop in continuous time	13
5.2 The impulse response, and the exponential miracle	13
5.3 When there is no closed form	14

6	Fitting from counts	15
6.1	The interval-censored log-likelihood (IC-LL)	15
6.2	Why KL and not least squares	15
6.3	In code	16
7	What you can and cannot learn	17
7.1	The κ - θ ridge	17
7.2	Standard errors under over-dispersion	17
7.3	Goodness of fit	18
8	Bayesian calibration and the identifiability ridge	20
8.1	The posterior	20
8.2	The Laplace approximation	21
8.3	Metropolis–Hastings, and why it samples the posterior	21
8.4	Hierarchical pooling: identification by borrowing strength	22
8.5	Bernstein–von Mises, and what Bayes can and cannot do	22
III	Bringing in covariates	24
9	Covariates in the baseline	25
9.1	The log-linear baseline	25
9.2	Fitting the effects	25
10	Covariates in the excitation: a Volterra system	27
10.1	The model	27
10.2	The equation is no longer a convolution	27
10.3	What survives: existence, and a time-varying ODE	28
10.4	In code, and a fast exact path	28
10.5	The general shape: Volterra quadrature	29
11	Multi-timescale kernels	30
11.1	Sum-of-exponentials kernels	30
11.2	A closed-form compensator via the matrix exponential	30
11.3	Fitting: a fixed bank of timescales	31
IV	Computation and learning	32
12	A zoo of solvers, and the operator viewpoint	33
12.1	The solvers, and when to use each	33
12.2	Rational Laplace transform $\Leftrightarrow$ finite ODE	34
12.3	The operator viewpoint	34
13	Learning the solver: neural operators, PINNs, PINOs	36
13.1	The cost we are attacking	36
13.2	Data-driven neural operators	36
13.3	Physics-informed solvers: PINN and PINO	37

13.4 Training the hard regime	37
V Practice	39
14 A guided tour of the package	40
14.1 Module map	40
14.2 Experiments	40
14.3 Reproducibility and tests	41
VI Appendices	42
A Volterra integral equations: a primer	43
A.1 What is it, and why “Volterra”?	43
A.2 The Neumann series and the resolvent	43
A.3 Existence and uniqueness — why Volterra is forgiving	44
A.4 Three solution strategies, by kernel type	44
B Transforms, ODEs, and state space	45
B.1 Laplace, Fourier, and the convolution theorem	45
B.2 From exponential kernel to a first-order ODE	45
B.3 Time-varying systems and the fractional case	46
C Estimation theory	47
C.1 IC-LL as a Bregman divergence	47
C.2 Consistency, information, goodness of fit	47
C.3 The κ - θ ridge, derived	48
C.4 Bernstein–von Mises: from likelihood to posterior	48
D API cheat-sheet	50
D.1 “I want to . . . ”	51
D.2 Parameter conventions	51
D.3 Further reading in the repository	52

Preface

This book is about a deceptively simple question: *how do you fit a self-exciting process when you never see the individual events, only how many happened in each time bucket?* Daily hospital admissions, hourly transaction volumes, per-day video views — in all of these the natural model is a Hawkes process (events trigger more events), yet the data are *counts*, not timestamps, and the Hawkes likelihood needs timestamps. The resolution we build, piece by piece, is the *Mean Behavior Poisson process* (MBPP): a companion process whose deterministic intensity equals the expected Hawkes intensity, which *can* be fit to counts and which shares its parameters with the Hawkes process we actually want.

Who this is for. A reader comfortable with calculus, linear algebra, and a little probability, who wants both the *mathematics* (stated carefully, with proofs) and the *code* (a working Python package, `hawkes_calibration`) — and, above all, the *reasons* behind each modelling and computational choice. Every chapter explains *why* we do what we do before showing *how*.

How to read it. The chapters build strictly in difficulty:

- **Part I** assembles the two ingredients — the Poisson process (and the one theorem that makes the whole approach possible) and the Hawkes process (and why it resists counts).
- **Part II** constructs the MBPP, solves its governing equation, fits it to counts, and asks what is actually learnable.
- **Part III** adds realism: covariates in the baseline, then — the hard case — covariates in the *excitation*, which turns the governing equation into a genuine Volterra system.
- **Part IV** is computation: the family of exact and numerical solvers, the operator viewpoint, and learning the solver with neural networks (DeepONet, FNO, PINN, PINO).
- **Part V** is a guided tour of the package and its experiments.
- The **appendices** are self-contained primers (Volterra equations, Laplace/Fourier, ODE/state-space, estimation theory) and an API cheat-sheet.

The boxes. Coloured boxes carry the pedagogy. *Why we care* motivates; *Intuition* gives the one-sentence mental model; *Why this choice* defends a design decision against alternatives; *Worked example* does a concrete computation; *Pitfall* flags a trap; and each chapter ends with a *recap*. Grey code listings show the actual implementation, and short exercises invite you to check understanding.

A note on honesty. We are candid about limitations throughout — which parameters are well identified and which are not, where a method is exact and where it is only an $O(\Delta t)$ approximation, and which models are easy versus hard to fit. Good modelling is as much about knowing the failure modes as the successes.

All code shown is from the companion package; every numerical claim in the book is reproduced by its test suite or experiments.

Part I

Foundations

Chapter 1

Events you cannot see one by one

Why we care

You manage a hospital and want to know whether infections are *self-propagating* (one case making the next more likely) or merely tracking an external driver. You have a clean record of *admissions per day* — but not who infected whom, nor the exact infection times. The model you want (a Hawkes process) is written in terms of event *times*; your data are *counts*. This chapter is about that mismatch and how we will dissolve it.

1.1 Three problems, one shape

Consider:

- **Epidemics.** Daily new cases or hospital admissions; individual infection times are unknown (privacy, or simply unrecorded).
- **Online popularity.** Daily views of a video, driven exogenously by shares/tweets and endogenously by word-of-mouth; platforms report daily volumes, not per-view timestamps.
- **Markets / operations.** Trades or tickets aggregated per minute or hour for storage or reporting.

In each case the data are *interval-censored*: time is partitioned into buckets $(o_{i-1}, o_i]$ and we observe only the count $C(o_{i-1}, o_i]$ in each. We would like to fit a self-exciting model and ask the same questions we could ask from timestamps: how much of the activity is self-generated, how fast does the excitation decay, how do covariates modulate it?

1.2 The model we want, and why it resists

The Hawkes process has conditional intensity (rate of events given the past)

$$\lambda(t) = \underbrace{s(t)}_{\text{exogenous}} + \underbrace{\sum_{t_i < t} \phi(t - t_i)}_{\text{endogenous}}, \quad (1.1)$$

where $s \geq 0$ injects *immigrant* events and the kernel $\phi \geq 0$ says each past event lifts the future rate, creating *offspring*. Two obstructions stop us fitting (1.1) to counts.

Why this choice: timestamps

The maximum-likelihood objective for a point process, $\ell(\theta) = \sum_{t_i \leq T} \log \lambda(t_i) - \int_0^T \lambda$, evaluates the intensity *at the event times* t_i . No times, no likelihood.

Why this choice: dependent increments

Even if we tried to model just the counts, the Hawkes process has *no independent-increments property*: the number of events in a bucket depends on the counts in earlier buckets (past events are still exciting the present). So we cannot treat the buckets as independent Poisson variables either — the trick that makes ordinary count models easy is unavailable.

1.3 The plan: fit a well-behaved twin

We will not fit the Hawkes process directly. Instead we build a *companion* process — the Mean Behavior Poisson process — whose intensity is the *expected* Hawkes intensity,

$$\xi(t) = \mathbb{E}[\lambda(t)], \quad (1.2)$$

and which, being an (inhomogeneous) Poisson process, *does* have independent increments and therefore a tractable count likelihood. Because ξ is built from the same s and ϕ as the Hawkes process, fitting the companion to counts recovers (approximations of) the Hawkes parameters.

Intuition

Stop trying to track each random realisation and track its *average* instead. The average intensity is deterministic; a process with a deterministic rate is Poisson; and Poisson counts are independent and easy. We trade a little fidelity (the average is not the process) for a likelihood we can actually write down.

1.4 What this book builds

By the end you will be able to: simulate self-exciting data with or without covariates (Ch. 3, 10); compute the MBPP intensity and compensator in closed form, by ODE, or by quadrature (Ch. 5, 12); fit the model from counts and quantify uncertainty (Ch. 6, 7); let covariates drive the baseline and even the excitation (Ch. 9, 10); and replace the solver with a learned neural operator when no closed form exists (Ch. 13). Crucially, you will know *when each tool is the right one* and *what each one cannot tell you*.

Chapter recap

Interval-censored counts break the Hawkes likelihood (no timestamps) and the Poisson shortcut (no independent increments). The fix is to fit a deterministic-mean companion process that restores both — the subject of Part II. The two ingredients we need first are the Poisson process (Ch. 2) and the Hawkes process (Ch. 3).

Exercise 1.1. Name a dataset from your own field that is interval-censored. What would the “immigrants” and “offspring” be, and why might the timestamps be unavailable?

Exercise 1.2. Explain in one sentence each, to a colleague, the two reasons the Hawkes likelihood cannot be used on counts.

Chapter 2

Poisson processes, intensity, and compensators

Why we care

Everything we build rests on one fact: a point process whose “rate forecast” is fixed in advance — not reacting to history — is a Poisson process, and Poisson counts are independent and easy. This chapter makes “rate forecast” precise (the *compensator*) and proves the fact (Watanabe). Keep this chapter close; we invoke it constantly.

2.1 Counting processes and conditional intensity

A simple point process produces times $t_1 < t_2 < \dots$; its counting process is $N(t) = \#\{i : t_i \leq t\}$, and its internal history is $\mathcal{H}_t = \sigma\{N(s) : s \leq t\}$. The *conditional intensity* is the instantaneous event rate given the past,

$$\lambda(t) = \lim_{h \downarrow 0} \frac{1}{h} \mathbb{E}[N(t+h) - N(t) \mid \mathcal{H}_{t-}], \quad (2.1)$$

so heuristically $\mathbb{E}[dN(t) \mid \mathcal{H}_{t-}] = \lambda(t) dt$.

2.2 The compensator

By the Doob–Meyer decomposition there is a unique predictable increasing process Λ (the *compensator*) with $\Lambda(0) = 0$ such that $M(t) = N(t) - \Lambda(t)$ is a local martingale; when Λ is absolutely continuous, $\Lambda(t) = \int_0^t \lambda$.

Intuition

$\Lambda(t)$ is the predictable “running forecast” of how many events should have happened by t ; λ is its instantaneous rate. Subtract the forecast from reality and what’s left, $M = N - \Lambda$, is pure surprise (a martingale). So λ is exactly the part of the future the past lets you anticipate.

2.3 The Poisson process and the theorem that runs the book

Definition 2.1. N is an (inhomogeneous) Poisson process with mean measure Λ if for disjoint sets $A_1, \dots, A_k$ the counts $N(A_i)$ are *independent* with $N(A_i) \sim \text{Poisson}(\Lambda(A_i))$.

Two consequences are used everywhere: independent increments, and $\mathbb{E}[N(t)] = \Lambda(t)$. The converse is the crux.

Theorem 2.2 (Watanabe). *A simple point process with a continuous compensator Λ is a Poisson process with mean measure Λ if and only if Λ is deterministic.*

Why this choice: this is the whole idea

Dependence between increments can only come from the intensity *reacting to history*. If the intensity — hence the compensator — is fixed in advance, there is no channel for the past to influence the future, so the process must be Poisson. This is precisely why, in Part II, we replace the random Hawkes intensity λ by its deterministic mean $\xi = \mathbb{E}[\lambda]$: doing so *manufactures* independent increments and hands us a product likelihood.

2.4 The likelihood (why it needs timestamps)

Theorem 2.3 (Jacod). *A point process with intensity λ has, on $[0, T]$, log-likelihood (up to a constant)*

$$\ell(\theta) = \sum_{t_i \leq T} \log \lambda(t_i; \theta) - \int_0^T \lambda(s; \theta) \, ds. \quad (2.2)$$

The first term rewards having forecast high intensity exactly where events landed; the second penalises total forecast mass. It manifestly needs the times t_i — the obstruction of Chapter 1.

Worked example: homogeneous Poisson

For constant λ on $[0, T]$ with n events, (2.2) is $n \log \lambda - \lambda T$; setting the derivative to zero gives the obvious MLE $\hat{\lambda} = n/T$. We will reuse this “rate = count / exposure” logic when we estimate the latent immigrant rate from interval-censored data (Ch. 6).

Chapter recap

The compensator is the predictable forecast of accumulated events; Watanabe says a deterministic compensator $\Leftrightarrow$ Poisson $\Rightarrow$ independent increments; and the point-process likelihood needs timestamps. Part II turns the first two facts into a method.

Exercise 2.1. Show that for a homogeneous Poisson process the inter-event times are i.i.d. Exponential. (We use the converse — the time-rescaling theorem — as a goodness-of-fit test in Ch. 7.)

Exercise 2.2. Why does Watanabe require the compensator to be *deterministic*, not merely continuous? Give a process with a continuous but random compensator that is not Poisson.

Chapter 3

The Hawkes process and its cluster representation

Why we care

The Hawkes process is the model we actually believe in: events beget events. Understanding it as a *branching* process — a forest of family trees — is what makes its mean computable (Ch. 4), its simulation easy (this chapter), and its failure to be Poisson obvious.

3.1 Definition

Model 3.1 (Hawkes process). The intensity is $\lambda(t) = s(t) + \int_0^{t-} \phi(t-u) dN(u) = s(t) + \sum_{t_i < t} \phi(t-t_i)$, with exogenous rate $s \geq 0$ and triggering kernel $\phi \geq 0$. The *exponential kernel* $\phi(t) = \kappa\theta e^{-\theta t}$ has $\int_0^\infty \phi = \kappa$ (the *branching ratio*) and decay rate θ .

Why this choice: the exponential kernel

We default to $\phi = \kappa\theta e^{-\theta t}$ because it is the one kernel whose self-convolutions telescope (Ch. 5), turning every later equation into a closed form or a tiny ODE. Heavy-tailed (power-law) kernels are more realistic for social media but cost us the closed form — a trade we revisit in Ch. 12.

3.2 The cluster (branching) representation

Theorem 3.2 (Hawkes–Oakes). *The Hawkes process equals the following branching construction: immigrants arrive as a Poisson process of intensity s ; each event at u independently spawns offspring as a Poisson process of intensity $\phi(\cdot - u)$ on (u, ∞) ; the process is the superposition of all immigrants and descendants.*

Intuition

Self-excitation, made literal: every event founds a family, and the process is a *forest of independent family trees* rooted at the immigrants. The mean number of direct children

per event is $\int \phi = \kappa$, so a family is a Galton–Watson tree with mean offspring κ : finite on average iff $\kappa < 1$ (*subcritical*), with expected total size $1/(1 - \kappa)$.

This picture is also the simulator: sample immigrants from s , then recursively sample each event’s offspring. The package does exactly this.

Listing 3.1: Cluster simulation (from `ic.simulate.py`).

```

1 def simulate_separable_hawkes(exogenous, kappa, theta, T, seed=None):
2     rng = np.random.default_rng(seed)
3     immigrants = sample_immigrants(exogenous, T, rng) # from s(t)
4     offspring = []
5     current = list(immigrants)
6     while current: # breadth-first over generations
7         nxt = []
8         for p in current:
9             n = rng.poisson(kappa) # mean kappa children per event
10            child = p + rng.exponential(1.0/theta, size=n) # Exp(theta) delays
11            child = child[child <= T]
12            offspring.extend(child); nxt.extend(child)
13        current = nxt
14    return np.sort(immigrants), np.sort(np.array(offspring))
    
```

Why this choice: keep immigrants and offspring separate

Returning the two streams separately lets us later reconstruct *any* of the six observation regimes (event-time vs. interval-censored, exogenous vs. endogenous) from one simulation — essential for the experiments in Ch. 6.

3.3 Why it is not Poisson, and the mean-intensity equation

Because λ depends on the realised history, the compensator $\Lambda = \int \lambda$ is *random*; by Watanabe (Thm 2.2) the Hawkes process is therefore not Poisson and has dependent increments. But its *mean* is deterministic, and satisfies a clean equation.

Proposition 3.3 (mean-intensity equation). $\xi(t) = \mathbb{E}[\lambda(t)]$ satisfies $\xi(t) = s(t) + \int_0^t \phi(t - u) \xi(u) du$.

Proof. Take $\mathbb{E}[\cdot]$ in Model 3.1 and use, by the tower property, $\mathbb{E}[dN(u)] = \mathbb{E}[\lambda(u)] du = \xi(u) du$ (Tonelli justifies the interchange). $\square$

Intuition

Averaging the *stochastic* feedback over all realisations turns it into *deterministic* feedback: “on average, today’s rate is the baseline plus the kernel-weighted echo of the average past.” This equation — a Volterra equation of the second kind (Appendix A) — is the protagonist of the entire book.

Worked example: total progeny and slow Monte Carlo

A subcritical cluster's size is Borel-distributed with mean $1/(1 - \kappa)$ and variance $\kappa/(1 - \kappa)^3$. Near criticality ($\kappa \rightarrow 1$) the variance explodes: a few giant cascades dominate, and Monte-Carlo estimates of means converge painfully slowly — a recurring theme when we validate near-critical fits.

Chapter recap

The Hawkes process is a forest of branching family trees; $\kappa = \int \phi$ is the mean offspring per event and $\kappa < 1$ is stability. It is not Poisson (random compensator), but its mean intensity solves the Volterra equation $\xi = s + \phi * \xi$. Solving and exploiting that equation is Part II.

Exercise 3.1. Verify by simulation that the mean offspring per immigrant is $\kappa/(1 - \kappa)$ for $\kappa = 0.6$. (The package test `test_total_progeny_equals_branching_sum` does this.)

Exercise 3.2. Explain why the cluster representation makes the *mean* intensity deterministic even though each realisation's intensity is random.

Part II

The Mean Behavior Poisson Process

Chapter 4

The Mean Behavior Poisson process

Why we care

Here is the pivot of the book. We define the companion process whose deterministic intensity is the expected Hawkes intensity, observe that it is therefore Poisson (so its counts are independent and easy), and note that it shares parameters with the Hawkes process we want. Everything afterwards is solving and exploiting one equation.

4.1 Definition and parameter correspondence

Definition 4.1 (MBPP). Given a Hawkes process with exogenous s and kernel ϕ , the Mean Behavior Poisson process is the inhomogeneous Poisson process whose intensity is the expected Hawkes intensity,

$$\xi(t) = \mathbb{E}[\lambda(t)] = s(t) + \int_0^t \phi(t-u) \xi(u) du. \quad (4.1)$$

The same two objects (s, ϕ) define both processes, so they are in *one-to-one parameter correspondence*: fit the MBPP, read off the Hawkes parameters.

Intuition

The MBPP is the Hawkes process with its randomness averaged out. Figure-2-style experiments confirm it: the mean of thousands of simulated Hawkes intensities lies right on top of the (deterministic) MBPP curve ξ .

4.2 Why it is Poisson — and why that is the point

ξ is deterministic, hence so is the compensator $\Xi(t) = \int_0^t \xi$; by Watanabe (Thm 2.2) the MBPP is a genuine Poisson process. Its bucket counts are therefore *independent* $\text{Poisson}(\Xi(o_{i-1}, o_i])$, and the joint law factorises:

$$\mathbb{P}(N(o_{i-1}, o_i] = C_i, i = 1, \dots, m) = \prod_{i=1}^m \frac{\Xi(o_{i-1}, o_i]^{C_i}}{C_i!} e^{-\Xi(o_{i-1}, o_i]}. \quad (4.2)$$

Why this choice: a deterministic mean buys a product likelihood

The Hawkes process failed precisely because its compensator is random (Ch. 1). Replacing λ by its mean ξ makes the compensator deterministic, which by Watanabe restores independent increments, which gives the product above — the interval-censored likelihood we will minimise (Ch. 6). One substitution converts an intractable problem into a sum of Poisson terms.

4.3 In code

The package’s MBPP object holds a kernel and an exogenous function and exposes the intensity and compensator (Ch. 5 explains how they are computed):

Listing 4.1: Constructing and evaluating an MBPP.

```

1 from hawkes_calibration import MBPP, ExponentialKernel, Constant
2 import numpy as np
3
4 mbpp = MBPP(ExponentialKernel(kappa=0.6, theta=0.8), Constant(2.0, horizon=40))
5 t = np.linspace(0, 30, 600)
6 xi = mbpp.intensity(t) # the deterministic mean intensity
7 Xi = mbpp.compensator(t) # its cumulative integral (the Poisson means)

```

Worked example: the MBPP is the mean

Experiment `exp5_mbpp_impulse.py` overlays ξ on the Monte-Carlo mean of thousands of Hawkes intensities for a single impulse and for rectangle, Dassios, and sine exogenous functions; the closed-form ξ matches the simulated mean to Monte-Carlo error ($\sim 2\text{--}6\%$). This is the empirical content of Definition 4.1.

Pitfall

The MBPP is *not* the Hawkes process — only its mean. It cannot reproduce the clustering/over-dispersion of real Hawkes counts, which is why interval-censored counts are over-dispersed relative to the MBPP’s Poisson assumption (Ch. 7). The parameters transfer; the higher moments do not.

Chapter recap

The MBPP is the inhomogeneous Poisson process with intensity $\xi = \mathbb{E}[\lambda] = s + \phi * \xi$. Being deterministic-mean it is Poisson (Watanabe), so its counts are independent and its likelihood is a product; and it shares parameters with the Hawkes process. Next: how to actually solve (4.1).

Exercise 4.1. Argue that if $\phi = 0$ (no excitation) the MBPP and the Hawkes process coincide and both are the inhomogeneous Poisson process of rate s .

Exercise 4.2. Why is the MBPP count distribution *narrower* (less dispersed) than the true Hawkes count distribution with the same parameters?

Chapter 5

Solving the MBPP equation

Why we care

To fit the MBPP we must evaluate its intensity ξ and compensator Ξ . The governing equation is a Volterra integral equation of the second kind. This chapter solves it three ways — a closed form (exponential kernel), an exact ODE, and a numerical scheme — and explains when each is appropriate. Appendix A is the from-scratch primer on Volterra equations if these are new to you.

5.1 A feedback loop in continuous time

Write the Volterra operator $(\mathcal{V}\xi)(t) = \int_0^t \phi(t-u)\xi(u) du$, so the MBPP equation (4.1) is $(\mathcal{I} - \mathcal{V})\xi = s$: an input s fed through a feedback loop with gain ϕ .

Intuition

Inverting $\mathcal{I} - \mathcal{V}$ is summing a geometric series of echoes: $\xi = s + \mathcal{V}s + \mathcal{V}^2s + \dots$. The n -th term is the input convolved with ϕ^{*n} — the n -th generation of offspring. On a finite horizon this always converges (Appendix A); globally it converges iff $\int \phi = \kappa < 1$ (subcriticality), and the total echo mass is $\kappa/(1 - \kappa)$ — exactly the mean offspring per event from Ch. 3.

5.2 The impulse response, and the exponential miracle

Because the loop is linear and time-invariant, it is characterised by its impulse response $E = \delta + h$ with resolvent kernel $h = \sum_{n \geq 1} \phi^{*n}$; then $\xi = E * s = s + h * s$ for *any* input.

Theorem 5.1 (exponential resolvent). *For $\phi(t) = \kappa\theta e^{-\theta t}$, the self-convolutions telescope and*

$$h(t) = \sum_{n \geq 1} \phi^{*n}(t) = \kappa\theta e^{(\kappa-1)\theta t}, \quad t > 0. \quad (5.1)$$

Why this choice: one timescale, one exponential

A single exponential kernel has one memory timescale, so its resolvent is again a single exponential and the response is first order. This is why the exponential kernel makes *everything* closed-form (and, in Ch. 12, a first-order ODE). It is the reason we default to it.

The compensator obeys the *same* equation with s replaced by its integral $S = \int_0^t s$, so $\Xi = S + h * S$, and over a bucket $\Xi(x, y] = \Xi(y) - \Xi(x)$.

Worked example: a single immigrant

For $s = \delta(\cdot - a)$, with $c = (\kappa - 1)\theta < 0$,

$$\xi(t) = \delta(t - a) + \kappa\theta e^{c(t-a)}\mathbb{I}[t > a], \quad \Xi^{\text{endo}}(t) = \frac{\kappa}{1-\kappa}(1 - e^{c(t-a)})\mathbb{I}[t > a],$$

and $\Xi^{\text{endo}} \rightarrow \kappa/(1-\kappa)$ as $t \rightarrow \infty$: one immigrant leaves a decaying wake whose total offspring mass is the branching sum. Every other exogenous shape (rectangle, piecewise-constant, sine, Dassios) is a superposition of such responses; the package derives each in closed form.

5.3 When there is no closed form

For a non-exponential kernel (power-law), the series $h = \sum \phi^{*n}$ has no closed form. The package then approximates h by a finite sum of numerical convolutions and forms $\xi = s + h * s$ on a grid:

Listing 5.1: Exact vs numerical MBPP (from `mbpp/core.py`).

```
1 MBPP(ExponentialKernel(0.6, 0.8), exo, method="closed") # exact, Theorem 5.x
2 MBPP(PowerLawKernel(0.5, 1.0), exo, method="numeric") # finite-convolution, O(dt)
```

Pitfall

The numerical solver converges only at first order, $O(\Delta t)$ (the intensity has a corner at $t = 0$). It is a genuine approximation — the paper’s “approximate MBPP” — and is deliberately less accurate than the closed form. Prefer `method="closed"` whenever the kernel allows it.

Chapter recap

The MBPP equation is a convolution Volterra equation; its solution is $\xi = s + h * s$ with resolvent $h = \sum \phi^{*n}$. For the exponential kernel $h = \kappa\theta e^{(\kappa-1)\theta t}$ (closed form, and a first-order ODE in Ch. 12); otherwise a first-order numerical scheme. The compensator solves the same equation with $S = \int s$. Next: turning Ξ into a likelihood and fitting.

Exercise 5.1. Derive Ξ^{endo} for the single impulse by integrating ξ^{endo} , and check the $t \rightarrow \infty$ limit equals $\kappa/(1-\kappa)$.

Exercise 5.2. Why does subcriticality $\kappa < 1$ appear only when we want the *global* resolvent, not for existence on a finite horizon? (See Appendix A.)

Chapter 6

Fitting from counts

Why we care

We can now solve for Ξ ; this chapter turns it into a likelihood and an estimator. We derive the interval-censored log-likelihood, show it is a Kullback–Leibler (Bregman) divergence between counts and compensators, explain why that — not least squares — is the right loss, and fit on synthetic data.

6.1 The interval-censored log-likelihood (IC-LL)

By Ch. 4 the buckets are independent $\text{Poisson}(\Xi_i)$ with $\Xi_i = \Xi(o_{i-1}, o_i]$. Taking $-\log$ of the product and dropping the θ -free constant $\sum_i \log C_i!$ gives

$$\mathcal{L}_{\text{IC}}(\theta) = \sum_{i=1}^m \Xi_i(\theta) - \sum_{i=1}^m C_i \log \Xi_i(\theta). \quad (6.1)$$

Intuition

It compares a *count* C_i to an *expected count* Ξ_i — both extensive quantities — as it should. Minimising it pushes the model’s predicted bucket means toward the observed bucket counts.

6.2 Why KL and not least squares

(6.1) equals (up to a constant) the generalised KL divergence $\text{KL}(\mathbf{C}, \Xi) = \sum_i C_i \log \frac{C_i}{\Xi_i} - (C_i - \Xi_i)$. Swapping the convex generator $x \log x$ for $\frac{1}{2}x^2$ gives instead the sum-of-squares loss $\sum_i (C_i - \Xi_i)^2$. The choice of divergence is the choice of noise model (the Bregman $\leftrightarrow$ exponential-family correspondence, Appendix C): $x \log x \Rightarrow \text{Poisson} \Rightarrow \text{KL}$; $\frac{1}{2}x^2 \Rightarrow \text{Gaussian} \Rightarrow \text{least squares}$.

Why this choice: KL is correctly heteroscedastic

A Poisson bucket has variance equal to its mean, so the statistically correct loss must weight bucket i by $1/\Xi_i$ — which KL does and least squares does not. When counts span orders of magnitude (bursts, near criticality), least squares is dominated by the loud buckets and destabilises; KL stays well behaved. Experiment **exp6** shows least squares running to the

boundary near criticality while IC-LL recovers cleanly. (Historically, the HIP method is least squares against the *intensity* rather than the compensator; the MBPP framework derives and corrects it — Appendix C.)

6.3 In code

Listing 6.1: Fitting the branching ratio and decay from counts.

```

1 from hawkes_calibration import (Constant, simulate_separable_hawkes,
2     interval_censor, uniform_obs_times, fit_mbpp_ic, MultiImpulse)
3
4 imm, off = simulate_separable_hawkes(Constant(5.0, 30.0), kappa=0.6, theta=0.8, T=30.0)
5 obs = uniform_obs_times(30.0, 30) # daily buckets
6 counts = interval_censor(off, obs) # the only data we "see"
7 res = fit_mbpp_ic(obs, counts, MultiImpulse(imm), loss="ic-ll", endogenous=True)
8 print(res.kappa, res.theta) # ~0.6, ~0.8

```

Worked example: recovery across scenarios

`exp6_ic_scenarios.py` fits (κ, θ) from counts across the six observation regimes and two parameter sets. The branching ratio is recovered tightly ($\hat{\kappa} \approx 0.60 \pm 0.03$ sub-critical, ≈ 0.95 near-critical); the decay θ is noisier — the subject of the next chapter.

Chapter recap

The IC-LL (6.1) is the negative log-likelihood of the MBPP’s independent Poisson buckets, and equals a KL divergence between counts and compensators. KL (not least squares) is correct because Poisson variance equals mean. Fitting recovers the kernel parameters — but, as we see next, not all of them equally well.

Exercise 6.1. Show (6.1) is minimised at $\Xi_i = C_i$ when the Ξ_i are unconstrained (the saturated model), and explain why the constrained minimiser differs.

Exercise 6.2. Generate counts from an MBPP and confirm IC-LL recovers κ ; then corrupt a few buckets with large outliers and compare IC-LL vs least-squares sensitivity.

Chapter 7

What you can and cannot learn

Why we care

A responsible fit reports not just point estimates but *which* of them to trust. This chapter explains, with the information geometry behind it, why the branching ratio κ is well identified from counts while the decay θ is not; how to get standard errors that are honest under the Hawkes over-dispersion; and how to check the fit at all (goodness-of-fit).

7.1 The κ – θ ridge

Write $\phi = \alpha e^{-\beta t}$ with $\alpha = \kappa\theta$, $\beta = \theta$. The data inform two nearly-orthogonal things: the *branching ratio* $\kappa = \alpha/\beta = \int \phi$ (how much: the offspring fraction, visible in counts and cross-correlations) and the *timescale* $1/\theta$ (how fast: visible only in timing).

Why this choice: counts hide the timescale

A bucket compensator $\Xi_i = \int_{o_{i-1}}^{o_i} \xi$ *integrates away* the within-bucket timing that carries θ . So $\partial \Xi_i / \partial \theta$ is small and sign-oscillating while $\partial \Xi_i / \partial \kappa$ is order-one and monotone: the Fisher information is ill-conditioned along the α/β -preserving ridge. As buckets coarsen, the θ -information $\rightarrow 0$ while κ persists.

Intuition

Counts tell you *how much* activity is self-generated (κ) but not *how fast* the echo decays (θ). Treat κ (and, later, covariate effects) as trustworthy; fix or prior-constrain θ from domain knowledge or finer data, and report it with wide intervals.

7.2 Standard errors under over-dispersion

The observed information (Hessian of the IC-LL) gives model-based SEs. But true Hawkes counts are *over-dispersed* relative to the MBPP's Poisson assumption (offspring clustering), so the naive SEs are too small. The package applies a *quasi-Poisson* correction — inflate by $\sqrt{\text{dispersion}}$:

Listing 7.1: Standard errors with the quasi-Poisson correction (from `interval_censored.py`).

```
1 se = _observed_information_se(objective, best.x, build_to_natural) # Hessian-1, delta
   method
2 disp = np.sum((counts - Xi_opt)**2 / Xi_opt) / (m - 2) # Pearson dispersion
3 se_kappa, se_theta = se[0]*np.sqrt(max(disp,1.0)), se[1]*np.sqrt(max(disp,1.0))
```

Worked example: coverage

On *model-correct* (MBPP-Poisson) data the 95% interval for κ covers the truth $\approx 94\%$ of the time — the SE computation is right. On genuine over-dispersed Hawkes data the naive SE undercovers (57%); the quasi-Poisson correction lifts it to $\approx 82\%$. The residual gap is serial clustering a scalar dispersion cannot fully capture — honest, and a reason to also run a goodness-of-fit check.

7.3 Goodness of fit

Two complementary checks (Appendix C, `gof.py`):

Time-rescaling (event-time). If the fitted compensator is right, the rescaled inter-event gaps $\Lambda(t_k) - \Lambda(t_{k-1})$ are i.i.d. $\text{Exp}(1)$; a Kolmogorov–Smirnov test against $\text{Exp}(1)$ detects misspecification.

Dispersion (interval-censored). The Pearson statistic $\sum_i (C_i - \Xi_i)^2 / \Xi_i$ divided by degrees of freedom should be near 1; > 1 flags over-dispersion (unmodelled clustering), $\gg 1$ a wrong model.

```
1 from hawkes_calibration import ks_test_exp1, time_rescaling_residuals, dispersion
2 D, p = ks_test_exp1(time_rescaling_residuals(events, compensator)) # event-time
3 phi = dispersion(counts, Xi, n_params=2) # interval counts
```

Worked example: GOF separates models

In `exp10_deep.py`, the dispersion under the *correct* kernel clusters near 1.9 (the inherent Hawkes over-dispersion), while a deliberately *wrong* kernel gives dispersion ≈ 67 — a decisive separation.

Pitfall

Never report $\hat{\theta}$ from interval-censored data as if it were as reliable as $\hat{\kappa}$. And a dispersion near 1 does not by itself prove the model — combine it with the time-rescaling test when timestamps are available.

Chapter recap

κ is well identified, θ weakly so (counts integrate away timing). Honest SEs need a quasi-Poisson inflation for Hawkes over-dispersion; goodness-of-fit uses time-rescaling (KS vs $\text{Exp}(1)$) and the Pearson dispersion. With Part II complete — model, solve, fit, diagnose — Part III adds covariates.

Exercise 7.1. Explain why coarser buckets make θ *less* identifiable but leave κ essentially unchanged.

Exercise 7.2. Simulate over-dispersed counts and verify the dispersion statistic exceeds 1; then confirm the quasi-Poisson SE widens accordingly.

Chapter 8

Bayesian calibration and the identifiability ridge

Why we care

Chapter 7 showed the painful truth: counts pin down κ but barely constrain θ , and a single standard error cannot express that. The Bayesian posterior can. It reports honest, possibly non-Gaussian uncertainty; exposes the κ - θ ridge as a *correlation*; lets us inject prior knowledge on the weak direction; and — through hierarchical pooling — borrows strength across series to sharpen estimates the data alone cannot.

8.1 The posterior

Place a prior $\pi(\theta)$ on the parameters $\theta = (\kappa, \vartheta, \dots)$. With the interval-censored likelihood of Chapter 6, $L(\theta) = \exp(-\mathcal{L}_{\text{IC}}(\theta))$ (a product of independent Poisson bucket terms), Bayes' theorem gives

$$p(\theta \mid \mathbf{C}) = \frac{L(\theta) \pi(\theta)}{\int L(\theta') \pi(\theta') d\theta'} \propto \exp(-\mathcal{L}_{\text{IC}}(\theta)) \pi(\theta). \quad (8.1)$$

Everything we need — point estimates, intervals, the joint shape — is a summary of (8.1).

Why this choice: priors on the unconstrained scale

We must keep $\kappa \in (0, 1)$ and $\vartheta > 0$. Rather than constrained priors with awkward Jacobians, we work in the *unconstrained* coordinates $u = (\text{logit } \kappa, \log \vartheta, \dots)$ and place independent Gaussian priors there. A $\mathcal{N}$ on $\text{logit } \kappa$ is a *logit-normal* prior on κ ; a $\mathcal{N}$ on $\log \vartheta$ is a *log-normal* on ϑ . The sampler then explores an unbounded space (where random-walk Metropolis behaves well), and we transform samples back. Because the prior is defined directly on u , no change-of-variables Jacobian is needed.

8.2 The Laplace approximation

A second-order Taylor expansion of the log-posterior about its mode (the MAP $\hat{u}$) gives the cheapest useful posterior. Writing $g(u) = \log p(u \mid \mathbf{C})$ and $\nabla g(\hat{u}) = 0$,

$$g(u) \approx g(\hat{u}) - \frac{1}{2}(u - \hat{u})^\top H(u - \hat{u}), \quad H = -\nabla^2 g(\hat{u}), \quad (8.2)$$

so $p(u \mid \mathbf{C}) \approx \mathcal{N}(\hat{u}, H^{-1})$. H is the observed information (Hessian of $\mathcal{L}_{\text{IC}}$) plus the prior's curvature; the MAP and H are exactly the quantities the frequentist fitter already computes, so Laplace is essentially free.

Intuition

Laplace says: “near its peak the posterior looks Gaussian, with width set by how sharply the data curve the log-likelihood.” Along the κ - θ ridge the curvature is tiny, so H^{-1} is huge in that direction — the Gaussian is a long ellipse, the linearised picture of the banana we see in MCMC.

8.3 Metropolis–Hastings, and why it samples the posterior

When the posterior is non-Gaussian (small data, near criticality), we sample it exactly with Markov-chain Monte Carlo. Random-walk Metropolis proposes $u' = u + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \Sigma)$, and accepts with probability $a = \min\{1, p(u')/p(u)\}$.

Proposition 8.1 (stationarity). *The Metropolis chain has the posterior p as its stationary distribution.*

Proof. With symmetric proposal $q(u' \mid u) = q(u \mid u')$, the transition density for $u \neq u'$ is $K(u', u) = q(u' \mid u) \min\{1, p(u')/p(u)\}$. Then $p(u)K(u', u) = q \min\{p(u), p(u')\} = p(u')K(u, u')$: *detailed balance* holds, so p is invariant. With an irreducible, aperiodic chain the law converges to p . $\square$

Why this choice: adaptive proposals

The proposal covariance Σ controls mixing: too small and the chain crawls, too large and it rarely accepts. We use Haario's *adaptive Metropolis* — after a warm-up, set $\Sigma = (2.38^2/d) \widehat{\text{Cov}}$ from the chain's own history (the $2.38^2/d$ is the asymptotically optimal RWM scaling). Adaptation must *diminish* (we freeze it / decay it) to preserve ergodicity. We run several chains and check the Gelman–Rubin $\hat{R} \approx 1$.

Listing 8.1: Bayesian fit: posterior over (κ, θ) from counts (from `bayes.py`).

```
1 from hawkes_calibration import fit_mbpp_bayes, GaussianPrior
2 res = fit_mbpp_bayes(obs, counts, exogenous, method="mcmc", n_chains=4)
3 print(res.summary()) # means, 95% credible intervals, R-hat, corr(kappa, theta)
4 # inject knowledge of the timescale: a tight prior on log(theta)
5 prior = GaussianPrior(means=[0.0, np.log(1.0)], sds=[1.5, 0.12])
6 res_t = fit_mbpp_bayes(obs, counts, exogenous, prior=prior) # tightens theta
```

Worked example: the ridge, made visible

On the standard synthetic problem (`exp11_bayes.py`) the posterior gives κ a 95% credible interval $[0.47, 0.57]$ (truth 0.5) but θ a broad $[0.37, 3.1]$, with posterior correlation $\text{corr}(\kappa, \theta) \approx -0.35$ — the ridge as a number. An informative prior on θ collapses its interval to $[0.79, 1.26]$; Laplace and MCMC agree on κ to 0.005; and $\hat{R} < 1.01$.

8.4 Hierarchical pooling: identification by borrowing strength

Suppose we observe J short series, each too weak to pin down its own κ_j . Model them hierarchically (with θ fixed as the weak nuisance):

$$\text{logit } \kappa_j \sim \mathcal{N}(\mu, \tau^2), \quad \mu \sim \mathcal{N}(\mu_0, s_0^2), \quad \log \tau \sim \mathcal{N}(0, 1), \quad (8.3)$$

with the per-series IC-LL likelihoods. The posterior *shrinks* each κ_j toward the population mean by an amount set by its own information.

Intuition

In the conjugate Gaussian caricature, if series j alone would estimate $\hat{u}_j$ with precision $1/\sigma_j^2$ and the population has precision $1/\tau^2$, the posterior mean is the precision-weighted average $\frac{\sigma_j^{-2}\hat{u}_j + \tau^{-2}\mu}{\sigma_j^{-2} + \tau^{-2}}$: an uninformative (large σ_j) series is pulled hard toward the population, a sharp one barely moves. **This is a genuine identifiability gain** — pooling extracts a population-level κ that no single short series could.

In `exp11` the pooled population branching ratio has 95% credible interval $[0.47, 0.59]$ from six short series each individually uninformative (`fit_mbpp_bayes_hierarchical`).

8.5 Bernstein–von Mises, and what Bayes can and cannot do

Theorem 8.2 (Bernstein–von Mises, informal). *Under the regularity of Theorem in Appendix C and a prior positive at θ_0 , as data accumulate the posterior concentrates and becomes asymptotically Gaussian, $p(\theta | \mathbf{C}) \approx \mathcal{N}(\hat{\theta}_{\text{MLE}}, \mathcal{I}^{-1})$; Bayesian credible sets then coincide with frequentist confidence sets.*

Pitfall

BvM needs the parameter to be *identified*. Along the κ – θ ridge the information $\mathcal{I}$ is near-singular, so the posterior does *not* concentrate in θ no matter how many buckets you collect — it relaxes to the prior in that direction. Bayes reports this honestly (a wide marginal); it does not manufacture information the counts lack. To learn θ you need finer time resolution, event times, or a genuine prior — not more buckets.

Chapter recap

The posterior $p(\theta | \mathbf{C}) \propto e^{-\mathcal{L}_{\text{IC}}(\theta)} \pi(\theta)$ gives honest uncertainty: Laplace for a fast Gaussian approximation, adaptive Metropolis for the exact (banana-shaped) posterior. Priors on the unconstrained scale keep parameters valid; an informative θ -prior tightens the weak direction;

hierarchical pooling sharpens many weak series at once. Bernstein–von Mises ties credible to confidence sets *where the model is identified* — which, for θ from coarse counts, is exactly where it is not.

Exercise 8.1. Show that placing a $\mathcal{N}(m, s^2)$ prior on $\log \theta$ induces a log-normal prior on θ with median e^m . What does $s \rightarrow \infty$ correspond to?

Exercise 8.2. Prove detailed balance for Metropolis–Hastings with a *non*-symmetric proposal q , where $a = \min\{1, \frac{p(u')q(u|u')}{p(u)q(u'|u)}\}$.

Exercise 8.3. In the conjugate Gaussian hierarchy, derive the shrinkage factor and explain why a short, noisy series is shrunk more.

Part III

Bringing in covariates

Chapter 9

Covariates in the baseline

Why we care

Real systems have known drivers — weather, day-of-week, a marketing spend, a tweet volume. The easiest place to let them in is the immigrant rate. This chapter shows that doing so changes *nothing* about the solution machinery, and fits the covariate effects from counts.

9.1 The log-linear baseline

Let covariates $X(t) \in \mathbb{R}^p$ enter through a log link that keeps the rate non-negative for any coefficients:

$$s(t) = \exp(\gamma_0 + \gamma^\top X(t)). \quad (9.1)$$

Why this choice: only the forcing changes

The MBPP equation $\xi = s + \phi * \xi$ is *linear in the forcing* s and its kernel ϕ is untouched. Equation (9.1) changes *what* we feed in, not the operator $\mathcal{I} - \mathcal{V}$. So the entire closed-form solution of Ch. 5 carries over verbatim — $\xi = E * s$, $\Xi = E * S$ — with no new mathematics.

Proposition 9.1. *If $X(t)$ is piecewise-constant then so is $s(t)$ (rate $\exp(\gamma_0 + \gamma^\top X_k)$ on interval k), so ξ and Ξ are the same closed-form piecewise-constant responses as in Ch. 5; only the per-interval rates are reparameterised.*

Intuition

A covariate constant on each bucket merely relabels the baseline height there. The self-excitation machinery neither knows nor cares that the height came from a regression rather than a free parameter, so the solver is unchanged.

9.2 Fitting the effects

We now optimise the IC-LL over (γ_0, γ) jointly with (κ, θ) . A time-bucketed table — one row per interval, with covariate columns and a count column — maps directly onto the fit.

Listing 9.1: Estimating covariate effects from a daily table.

```

1 from hawkes_calibration import PiecewiseConstantCovariate, fit_mbpp_ic_covariates
2 breaks = np.arange(n_days + 1) # interval endpoints
3 cov = PiecewiseConstantCovariate(breaks, X) # X: (n_days, p) covariate columns
4 res = fit_mbpp_ic_covariates(breaks, counts, cov, loss="ic-ll")
5 res.gamma0, res.gamma, res.kappa, res.theta # recovered from counts + covariates

```

Worked example: regime-switching baseline

`exp8_ic_covariates.py` drives the baseline with a regime indicator that flips every few buckets. Over repeats the covariate effect is recovered tightly ($\gamma_1 = 0.90$ as 0.90 ± 0.06 from counts alone), even though θ and the absolute level γ_0 scatter — the κ - θ ridge of Ch. 7 again.

Why this choice: effects are identified by contrast

γ is read off from how counts change *across* buckets as X varies; the absolute split between baseline level and self-excitation is weaker. A covariate that does not vary carries no information about its coefficient: identification of γ needs covariate *contrast*.

Chapter recap

Baseline covariates enter through a log link and change only the forcing, so the closed-form solver is untouched; we fit γ jointly with (κ, θ) and recover the *effects* tightly (given contrast). Crucially, the kernel is still a fixed convolution. The next chapter asks what happens when covariates enter the *kernel* — and there the convolution structure finally breaks.

Exercise 9.1. Argue that with one-hot encoded categorical covariates, (9.1) gives a separate baseline level per category, and the fit reduces to estimating those levels plus (κ, θ) .

Exercise 9.2. Why can the covariate-effect γ be well identified even when the absolute baseline γ_0 is not?

Chapter 10

Covariates in the excitation: a Volterra system

Why we care

Sometimes the *contagiousness* itself varies — a disease spreads faster in winter, a post is more re-shareable during a trend. Then covariates must modulate the *kernel*, not just the baseline. This is the genuinely hard case, and the reason the whole Volterra viewpoint was worth developing: the governing equation stops being a convolution. We show it is still solvable, exactly how, and at what cost.

10.1 The model

Model 10.1 (covariate-modulated excitation). Let covariates $Z(t) \in \mathbb{R}^q$ scale the triggering strength,

$$\alpha(t) = \kappa \theta \exp(\delta^\top Z(t)), \quad \phi(t, u) = \alpha(t) e^{-\theta(t-u)}, \quad (10.1)$$

so the time-varying branching ratio is $\kappa(t) = \kappa e^{\delta^\top Z(t)}$.

10.2 The equation is no longer a convolution

Averaging as before gives

$$\xi(t) = s(t) + \int_0^t K(t, u) \xi(u) du, \quad K(t, u) = \kappa \theta e^{\delta^\top Z(t)} e^{-\theta(t-u)}, \quad (10.2)$$

a *general* (non-convolution) Volterra equation: $K(t, u)$ depends on t and u *separately*, not only on $t - u$.

Why this choice: what we lose, and why

A convolution kernel means the system reacts to a stimulus the same way whenever it arrives (time-invariance), which is exactly what let Laplace/Fourier diagonalise it and gave a single transfer function (Ch. 5, Ch. 12). Once the gain rides on $Z(t)$, *when* matters: no convolution theorem, no constant transfer function, and the spectral/operator shortcuts no longer apply.

Intuition

The same past event echoes more loudly in a high- Z period than a low- Z one. The “filter” is the same shape, but its volume knob is being turned over time.

10.3 What survives: existence, and a time-varying ODE

Well-posedness survives. (10.2) is a standard linear Volterra equation; by the general theory (Appendix A, which never used convolution) it has a unique solution on every finite horizon. Existence does not need time-invariance.

Exponential shape $\Rightarrow$ a finite LTV ODE. The kernel is *separable*: $K(t, u) = \underbrace{\kappa\theta e^{\delta^\top Z(t)}}_{a(t)} \underbrace{e^{-\theta t}}_{u \text{ only}} e^{\theta u}$.

A separable Volterra kernel collapses to finitely many ODEs (Appendix A). With the filtered history $y(t) = \int_0^t e^{-\theta(t-u)} \xi(u) du$,

$$y'(t) = \xi(t) - \theta y(t), \quad \xi(t) = s(t) + \kappa\theta e^{\delta^\top Z(t)} y(t), \quad (10.3)$$

a *linear time-varying* (LTV) ODE: the same state as the convolution case, but with a gain that breathes with the covariate.

Intuition

“Same state, time-varying gain.” We keep a finite-dimensional, integrable system; we merely lose the constant-coefficient (hence transform-solvable) structure.

10.4 In code, and a fast exact path

The general solver is the LTV integrator `solve_mbpp_ltv`; for a piecewise-constant covariate the package uses an *exact per-regime* propagation (the gain is constant on each regime, so the 1-D linear ODE is solved in closed form) — fast and more accurate than a grid.

Listing 10.1: Forward solve and fitting with excitation covariates.

```
1 from hawkes_calibration import solve_mbpp_ltv, fit_mbpp_ic_excitation
2 # forward: xi for covariate-modulated excitation
3 xi = solve_mbpp_ltv(forcing, A0=[[kappa*theta]], B=[[theta]], t_grid,
4                     modulation=lambda t: [[np.exp(delta @ Z(t))]])
5 # fit delta (the excitation-covariate effect) from interval counts:
6 res = fit_mbpp_ic_excitation(obs_times, counts, Z) # recovers kappa, theta, mu, delta
```

Worked example: recovering δ

With $Z \equiv 0$ the LTV solver reproduces the constant-kernel solver to 10^{-16} ; a covariate that doubles excitation over a window elevates the intensity then relaxes back. On MBPP-Poisson data the fitter recovers $\delta = 1.2$ as ≈ 1.14 in ~ 0.5 s (the exact per-regime path); on genuine over-dispersed Hawkes data (`exp10`) it is recovered on average (1.49 ± 0.9) over groups).

Pitfall

Excitation covariates make the IC-LL *non-convex* (the δ enter through $e^{\delta^\top Z}$ inside the intensity), and δ is weakly identified from little data. Use many sequences and restarts; identification needs the covariate to vary *while events occur*. Multivariate excitation covariates add up to pM^2 parameters and are harder still — the supported fitter is univariate.

10.5 The general shape: Volterra quadrature

If the triggering shape is not a finite sum of exponentials (e.g. power-law), the kernel is not separable and no finite ODE exists. The package then solves (10.2) directly by trapezoid-Nyström quadrature (`solve_mbpp_volterra`, Appendix A) — $O(N^2)$ and first-order, but fully general.

Chapter recap

Excitation covariates turn the MBPP equation into a general Volterra equation: well-posed, but no longer time-invariant, so the transform shortcuts vanish. For an exponential shape it reduces to a finite LTV ODE (`solve_mbpp_ltv`, fast exact per-regime path); for a general shape, to Volterra quadrature. Fitting recovers δ but is non-convex and data-hungry. This is the precise content of “excitation covariates are harder.”

Exercise 10.1. Show that placing the modulation at the *parent* time, $K(t, u) = \kappa\theta e^{\delta^\top Z(u)}e^{-\theta(t-u)}$, is also separable, and write the corresponding LTV ODE.

Exercise 10.2. Why does the spectral (Fourier) operator of Ch. 12 fail to represent (10.2) exactly, whereas it was exact for the convolution case?

Chapter 11

Multi-timescale kernels

Why we care

Real excitation often has *several* timescales: a fast burst of immediate reactions and a slow tail of long-term influence. A sum of exponentials captures this, stays inside our closed-form machinery, and — with a fixed bank of decay rates — is well-conditioned to fit.

11.1 Sum-of-exponentials kernels

Replace the single exponential by

$$\phi(t) = \sum_{q=1}^Q \kappa_q \theta_q e^{-\theta_q t}, \quad \text{branching ratio} = \sum_q \kappa_q. \quad (11.1)$$

Every result of Ch. 5 still holds per component, and the state-space ODE (Ch. 12) simply carries Q states instead of one.

11.2 A closed-form compensator via the matrix exponential

For a constant baseline the compensator has an exact closed form. The state vector $u \in \mathbb{R}^Q$ obeys $\dot{u} = Au + a\mu$ with $A = a\mathbf{1}^\top - \text{diag}(b)$, $a_q = \kappa_q \theta_q$, $b_q = \theta_q$, and $\xi = \mu + \mathbf{1}^\top u$. Diagonalising $A = V\Lambda V^{-1}$ gives, with $w = A^{-1}a$ and $p_i = (\mathbf{1}^\top V)_i (V^{-1}w)_i$,

$$\Xi(t) = \mu \left(t + \sum_i p_i \left(\frac{e^{\lambda_i t} - 1}{\lambda_i} - t \right) \right). \quad (11.2)$$

Why this choice: why this matters for speed

The pure-Python ODE-in-a-loop is too slow to call thousands of times inside an optimiser. This matrix-exponential formula evaluates the compensator at all bucket endpoints in $O(Q^3 + mQ)$ with *no* time grid — turning a multi-minute fit into a fraction of a second (Ch. 13 discusses the general remedy). A is invertible because $\sum_q \kappa_q < 1$ makes it Hurwitz.

11.3 Fitting: a fixed bank of timescales

Why this choice: fix the decays, fit the weights

Estimating both the rates θ_q and weights κ_q is ill-posed (timescales are weakly identified — Ch. 7, now $\times Q$). The standard, well-conditioned remedy is to fix a *bank* of decay rates (fast, medium, slow) and fit only the non-negative weights $\kappa_q \geq 0$, optionally with an ℓ_1 penalty to select a sparse set of active timescales.

Listing 11.1: Multi-timescale fit over a fixed decay bank.

```
1 from hawkes_calibration import fit_mbpp_ic_sumexp
2 bank = np.array([0.4, 1.0, 2.0, 5.0]) # fixed decay rates (slow ... fast)
3 res = fit_mbpp_ic_sumexp(obs, counts, bank, l1=0.02) # fit non-negative weights
4 res.kappas, res.thetas # weight per timescale; total = sum(kappas)
```

Worked example: total vs. split

With a true 2-timescale kernel and a 3-rate bank, the fit recovers the *total* branching ratio well (0.58 vs. 0.60) but spreads the weights across the bank — the per-timescale split is weakly identified, exactly as the κ - θ analysis predicts. Report the total and the rough fast/slow balance, not a precise decomposition.

Chapter recap

Sum-of-exponentials kernels model fast+slow excitation, stay closed-form (matrix exponential for the compensator), and are fit by fixing a decay bank and estimating non-negative weights (with ℓ_1). The total branching ratio identifies well; the timescale split does not.

Exercise 11.1. Show that a single-exponential kernel is the $Q = 1$ case and recovers Ch. 5. Why does the matrix A have all eigenvalues with negative real part when $\sum_q \kappa_q < 1$?

Part IV

Computation and learning

Chapter 12

A zoo of solvers, and the operator viewpoint

Why we care

We now have several ways to compute ξ and Ξ . This chapter organises them into a decision guide — which solver for which model — and then steps back to the unifying picture: the MBPP map $s \mapsto \xi$ is a linear operator, which is what lets us, in the next chapter, *learn* it.

12.1 The solvers, and when to use each

Situation	Method	Function	Status
exp. kernel, any baseline	closed-form impulse response	<code>MBPP(method="closed")</code>	exact
general kernel (power-law)	finite convolution sum	<code>MBPP(method="numeric")</code>	$O(\Delta t)$
sum-of-exp, any forcing	state-space ODE	<code>solve_mbpp_ode</code>	exact
multivariate, exp.	M^2 -state ODE	<code>solve_mbpp_ode_multivariate</code>	exact
excitation covariates (exp.)	linear time-varying ODE	<code>solve_mbpp_ltv</code>	exact (per-regime)
sum-of-exp, const. baseline	matrix exponential	<code>_sumexp_compensator_const</code>	exact
<i>any</i> non-separable kernel	Volterra–Nyström quadrature	<code>solve_mbpp_volterra</code>	$O(\Delta t)$

Why this choice: exact beats approximate when you can

Prefer a closed form or an exact ODE whenever the kernel allows; fall back to Nyström quadrature only for genuinely non-separable kernels (power-law + excitation covariates). The exact solvers are both faster and free of discretisation error, and — crucially for fitting — give clean gradients.

12.2 Rational Laplace transform $\Leftrightarrow$ finite ODE

The bridge between the integral equation and the ODE solvers:

Theorem 12.1 (realisation). *The convolution MBPP equation is equivalent to a finite linear constant-coefficient ODE iff ϕ has a strictly proper rational Laplace transform — equivalently, iff ϕ is a finite sum of (polynomial $\times$) exponentials.*

Intuition

A memory built from finitely many decaying modes can be tracked by finitely many state variables: rational transfer function $\Leftrightarrow$ finite-dimensional machine. A power-law memory has infinitely many timescales, so no finite ODE exists — hence the quadrature fallback (and, for the curious, the Mittag-Leffler / fractional treatment in Appendix C).

12.3 The operator viewpoint

The solution map $G : s \mapsto \xi$ is *linear and translation-invariant*. In the Fourier domain it is a pointwise multiply,

$$\hat{\xi}(\omega) = R(\omega) \hat{s}(\omega), \quad R(\omega) = \frac{1}{1 - \hat{\phi}(\omega)}, \quad (12.1)$$

the *transfer function*. For the exponential kernel $R(\omega) = (\theta + i\omega)/((1 - \kappa)\theta + i\omega)$ — a one-pole filter whose pole is the ODE rate. Time domain, frequency domain, and state space are three views of one operator.

Why this choice: a spectral neural operator is exact here

Because G is diagonalised by the Fourier transform, a single linear Fourier-neural-operator layer with weights $R(\omega)$ is the exact MBPP operator. Conversely, *learning* $R(\omega)$ from input/output pairs is nonparametric identification of the operator (and of $\phi = \mathcal{F}^{-1}\{1 - 1/R\}$) — the Wiener–Hopf idea in neural-operator clothing. This is the springboard for Ch. 13. (Note: this exactness relies on translation invariance, which the excitation-covariate model of Ch. 10 breaks.)

Listing 12.1: The operator backends (from `operators/linear.py`).

```
1 from hawkes_calibration import FunctionalMBPP, SpectralOperator
2 FunctionalMBPP(kernel, method="ode").solve(forcing, t_grid) # state-space ODE
3 FunctionalMBPP(kernel, method="spectral").solve(forcing, t_grid) # exact R(omega)
4 SpectralOperator(t_grid).fit(S, XI).recover_kernel() # LEARN R, recover phi
```

Chapter recap

The solvers form a ladder from exact closed form to general $O(\Delta t)$ quadrature; rational Laplace transform $\Leftrightarrow$ finite ODE tells you which rung you are on. Underneath, the MBPP map is a linear operator, exact as one spectral layer — the bridge to learning the solver.

Exercise 12.1. Place the multi-timescale and excitation-covariate models on the solver table and justify the choice.

Exercise 12.2. Derive $R(\omega)$ for the exponential kernel from $\hat{\phi}(\omega) = \kappa\theta/(\theta + i\omega)$ and identify its pole.

Chapter 13

Learning the solver: neural operators, PINNs, PINOs

Why we care

For a non-closed-form model (power-law + excitation covariates), fitting calls a slow Python solve thousands of times, and finite-difference gradients multiply the cost by $2n_{\text{params}}$. The fix is to *learn* the forward map with a neural network: one fast, autodifferentiable forward pass replaces the loop, and autodiff replaces the finite differences. This chapter explains the ladder of such methods and the trade-offs.

13.1 The cost we are attacking

$$\text{cost} \approx (\text{restarts}) \times (\text{iters}) \times (1 + 2n_{\text{params}}) \times (\text{one Python solve}).$$

Two multiplicative culprits: the Python time-stepping loop, and the $2n_{\text{params}}$ finite-difference gradient. Compiling the loop (Numba/JAX) kills the first; automatic differentiation (and the adjoint method) kills the second. A learned solver does both at once.

13.2 Data-driven neural operators

A *neural operator* learns a map between function spaces, $G : s \mapsto \xi$, from (input, output) pairs produced by the exact solver. The package provides three (numpy, and TensorFlow at scale):

- **DeepONet** — branch encodes the forcing at sensor points, trunk encodes the query time; output $\xi(t) = \sum_k b_k(s) \tau_k(t)$. Drops the linearity assumption, so it extends to no-closed-form regimes.
- **Fourier Neural Operator (FNO)** — spectral convolution layers; the natural architecture because the linear MBPP operator is *diagonal* in Fourier (Ch. 12); a single linear layer is exact.
- **Amortised inference** — learns the *inverse* map $\text{counts} \mapsto (\kappa, \theta)$, so fitting a new series is one forward pass.

Worked example: what the operators recover

In `exp9`, the learned spectral operator generalises to new forcings at $\sim 5\%$ L2 and recovers the branching ratio (0.64 vs. 0.6); the DeepONet reaches $\sim 12\%$ (a general learner, no structure exploited); the amortised net predicts κ at correlation 0.93 in one pass but only ~ 0.1 for the weakly-identified θ — the identifiability of Ch. 7 showing through.

13.3 Physics-informed solvers: PINN and PINO

A *physics-informed* network needs no ground-truth solver: it is trained to make the MBPP residual vanish.

Why this choice: train on the residual, not on data

For a network $N(t, p) \approx \xi(t; p)$, the loss is the collocation residual $R(t) = N(t) - s(t) - \int_0^t K(t, u)N(u)du$ (quadrature) plus the initial condition. Because this residual is *linear* in ξ , its global minimum is provably the exact solution (verified in numpy: `solve_mbpp_residual_linear` matches the time-stepping solve to 10^{-9}). So “minimise the residual” is exactly “solve the equation,” and no exact solver is needed.

- **Parametric PINN** (`mode="pinn"`) — $N(t, p)$ generalises over the scalar parameters $p = (\kappa, \theta, \mu, \delta)$ for a *fixed* functional setting. Change the covariate path and you retrain.
- **Physics-Informed Neural Operator, PINO** (`mode="pino"`) — a DeepONet whose branch ingests the *covariate path itself*, trained on the residual over a *distribution* of covariate paths *and* parameters. One operator then solves any instance in the family in a single forward pass.

Intuition

PINN generalises over a parameter slice; PINO generalises over the whole family of *functions*. The PINO is “a PINN trained on the family” — the right object when the covariate path, not just the parameters, varies.

Listing 13.1: JAX/TensorFlow PINN or PINO behind one interface (from `neural_solver.py`).

```

1 from hawkes_calibration.operators.neural_solver import make_neural_solver
2 op = make_neural_solver(backend="jax", mode="pino", coll_grid=t, T=60.0)
3 op.train(n_steps=30000, batch=32) # one-off; no ground-truth solver needed
4 xi = op.solve(dict(kappa=0.4, theta=1.0, mu=2.0, delta=[1.2]), t, Z_on_grid=Z_new)

```

13.4 Training the hard regime

A PINO over a wide family is hardest near criticality ($\kappa \rightarrow 1$, stiff ξ). Three stabilisers (all in the trainers):

- **hybrid loss** — add a few *exact* anchors (`make_anchor_data`) as a supervised term weighted by `data_weight`;

- **curriculum** — sample κ from an easy range first, widen toward criticality;
- **residual re-weighting** — normalise by the solution scale so stiff cases don't dominate.

The diagnostic `solver_accuracy_report` compares the trained solver to the exact MBPP across a parameter grid and bins error by κ , showing exactly where it breaks.

Pitfall

A learned solver is only as good as its training range: it will extrapolate poorly outside the sampled $(\kappa, \theta, \delta, Z)$ box, and worst near criticality. Always run the accuracy report before trusting it inside a fit, and prefer the exact solvers whenever the model admits one.

Chapter recap

Learned solvers attack both fitting bottlenecks: a fast forward pass and autodiff gradients. Data-driven operators (DeepONet, FNO, amortised) learn from solver output; physics-informed solvers (PINN, PINO) learn from the residual itself, with the PINO generalising over the whole functional family. Hybrid anchors, a curriculum, and re-weighting tame the stiff regime; the accuracy report tells you where to trust the result.

Exercise 13.1. Why is a single linear FNO layer *exact* for the convolution MBPP but only approximate for the excitation-covariate model?

Exercise 13.2. Explain how the hybrid (residual + anchor) loss helps precisely where the residual alone is weak.

Part V

Practice

Chapter 14

A guided tour of the package

Why we care

Theory is only useful if you can run it. This chapter maps the mathematics of the book onto the `hawkes_calibration` package: which module does what, which experiment reproduces which figure, and how the test suite backs every numerical claim.

14.1 Module map

Module	Role (chapter)
<code>eventtime/</code>	event-time Hawkes MLE: simulate, likelihood, estimate, lasso, covariates (Ch. 4,5)
<code>mbpp/core.py</code>	MBPP intensity/compensator, closed & numeric (Ch. 4,5)
<code>mbpp/exogenous.py</code>	baseline functions incl. covariate link (Ch. 5,9)
<code>mbpp/ic_simulate.py</code>	cluster & excitation-modulated simulators (Ch. 3,10)
<code>mbpp/interval_censored.py</code>	IC-LL/SSE, all fitters, SEs, forecasting (Ch. 6–11)
<code>mbpp/gof.py</code>	time-rescaling KS, dispersion (Ch. 7)
<code>mbpp/bayes.py</code>	Bayesian posterior, priors, hierarchical pooling (Ch. 8)
<code>operators/linear.py</code>	ODE/state-space, Volterra, spectral operator (Ch. 12)
<code>operators/nn.py</code>	numpy DeepONet, amortised inference (Ch. 13)
<code>operators/tf.py</code>	TensorFlow high-dim FNO/DeepONet/SSM (Ch. 13)
<code>operators/neural_solver*.py</code>	PINN/PINO in JAX & TensorFlow (Ch. 13)

14.2 Experiments

Each experiment is runnable (`python -m experiments.expN...`) and writes a figure to `results/`:

- `exp1--4` — event-time Hawkes: bias/variance, elicitation-matrix recovery, covariates, sparse high-dimensional lasso.
- `exp5` — MBPP = mean Hawkes intensity (Ch. 4).

- `exp6` — recover (κ, θ) from counts across scenarios; IC-LL vs. SSE (Ch. 6).
- `exp7` — interval-censored forecasting (ACTIVE-style).
- `exp8` — baseline covariates (Ch. 9).
- `exp9` — operator backends: ODE/spectral/DeepONet/amortised (Ch. 12,13).
- `exp10` — excitation covariates on true Hawkes data, multi-timescale, goodness-of-fit (Ch. 10,7).

14.3 Reproducibility and tests

Every numerical claim in the book is checked by the test suite (`tests/test_interval_censored.py`, `test_operators.py`, `test_deep.py`, `test_neural_solver.py`) — e.g. the MBPP equals the Monte-Carlo Hawkes mean, the closed-form and numerical solvers agree, the IC-LL recovers κ , the Volterra and LTV solvers agree where both apply, the family residual is exactly zero, and the standard errors cover at the nominal rate on model-correct data. The package **README** consolidates the practical reference — the capability matrix, the learned-solver (PINN/PINO) recipe, and the TensorFlow noise-stress harness — alongside this book.

Why this choice: tests as the book’s ground truth

Where the text asserts a number (“ $\hat{\kappa} \approx 0.60 \pm 0.03$ ”, “residual = 0 to 10^{-9} ”), there is a test that produces it. Treat the suite as the executable version of this book.

Chapter recap

The package mirrors the book: simulation and MLE for the event-time case; the MBPP, its solvers, and its fitters for counts; covariate and multi-timescale extensions; the operator and neural-solver machinery; and goodness-of-fit. Experiments reproduce the figures and tests certify the claims.

Exercise 14.1. Run `exp6` and `exp10` and match each panel to a chapter result.

Exercise 14.2. Pick one theorem from Part II and find the test that numerically verifies it.

Part VI

Appendices

Appendix A

Volterra integral equations: a primer

This appendix is self-contained and assumes no prior exposure to integral equations. Everything the main text uses about Volterra equations is here.

A.1 What is it, and why “Volterra”?

A *linear Volterra equation of the second kind* is

$$\xi(t) = s(t) + \int_0^t K(t, u) \xi(u) du, \quad t \in [0, T], \quad (\text{A.1})$$

with s (forcing) and K (kernel) given and ξ unknown. “Second kind” means the unknown also appears outside the integral (the lone $\xi(t)$); a *first-kind* equation $s = \int_0^t K \xi$ has it only inside and is typically ill-posed (a deconvolution). “Volterra” refers to the upper limit being the running time t : the integral looks only at the *past* $u \leq t$ (contrast Fredholm, $\int_0^T$). That causal, “lower-triangular in time” structure is the source of all the good behaviour below.

Intuition

Read (A.1) as a feedback rule unrolling in time: $\xi(t)$ now equals an external input $s(t)$ plus a weighted echo of all of ξ ’s past, with weight $K(t, u)$. Because the echo only reaches backward, you can sweep forward and build ξ left to right — nothing in the future is needed.

A.2 The Neumann series and the resolvent

With the Volterra operator $(\mathcal{K}\xi)(t) = \int_0^t K(t, u)\xi(u) du$, equation (A.1) is $(\mathcal{I} - \mathcal{K})\xi = s$, formally inverted by the geometric (Neumann) series $\xi = \sum_{n \geq 0} \mathcal{K}^n s$. The sum $R(t, u) = \sum_{n \geq 1} K_n(t, u)$ of the iterated kernels is the *resolvent*, and $\xi(t) = s(t) + \int_0^t R(t, u)s(u) du$: solve for R once, then one integral per forcing.

Intuition

$\mathcal{K}s$ is the first-generation echo, $\mathcal{K}^2 s$ the echo of the echo, and the Neumann series sums all generations. In the Hawkes setting R is literally the renewal density of the branching tree (Ch. 3).

A.3 Existence and uniqueness — why Volterra is forgiving

Theorem A.1. *If $|K| \leq \bar{K}$ on $0 \leq u \leq t \leq T$ and $s \in C[0, T]$, then (A.1) has a unique continuous solution on $[0, T]$, the (uniformly convergent) Neumann series.*

Proof. By induction $|(\mathcal{K}^n \xi)(t)| \leq \bar{K}^n \|\xi\|_\infty t^n / n!$ (each nested integration over a shrinking triangle contributes a $1/n!$). Hence $\|\mathcal{K}^n\| \leq (\bar{K}T)^n / n! \rightarrow 0$, $\sum_n \mathcal{K}^n$ converges in operator norm, and $(\mathcal{I} - \mathcal{K})^{-1}$ exists. $\square$

Why this choice: the factorial saves you

The decisive estimate is $\bar{K}^n t^n / n!$: the factorial overwhelms any fixed gain, so on a finite horizon the loop is *always* invertible — *no* smallness condition on the kernel. This is why the MBPP equation is solvable even when the excitation is covariate-modulated and time-varying (Ch. 10): the proof never used convolution structure. A smallness condition ($\int \phi = \kappa < 1$) appears only for the *global* resolvent on $[0, \infty)$.

A.4 Three solution strategies, by kernel type

(i) **Convolution** $K(t, u) = \phi(t - u) \Rightarrow$ **transforms**. The system is time-invariant; Laplace/-Fourier give $\hat{\xi} = \hat{s} / (1 - \hat{\phi})$ (Appendix B), and the resolvent is itself a convolution (a single exponential for the exponential kernel). *Ch. 5.*

(ii) **Separable** $K(t, u) = \sum_r a_r(t) b_r(u) \Rightarrow$ **ODE**. Finite rank collapses (A.1) to ODEs: with $y_r = \int_0^t b_r \xi$, $\xi = s + \sum_r a_r y_r$ and $y'_r = b_r(t) \xi$. Constant a_r, b_r give a constant-coefficient ODE; a t -dependent a_r (the covariate factor $e^{\delta^\top Z(t)}$) gives a *time-varying* one. *This is why Ch. 10 reduces to the LTV system.*

(iii) **General** $\Rightarrow$ **quadrature (Nyström)**. Discretise the integral on a grid; (A.1) becomes a lower-triangular linear system solved by forward substitution in $O(N^2)$ (`solve_mbpp_volterra`). First-order accurate; the fallback for power-law + excitation covariates.

Intuition

Same idea — invert $\mathcal{I} - \mathcal{K}$ — specialised to how much structure K has: maximum (convolution) $\rightarrow$ a formula; medium (finite rank) $\rightarrow$ a few ODEs; none $\rightarrow$ march forward in time, which the causal form always allows.

Pitfall

The inverse problem — recovering ϕ from observed mean behaviour — is a *first-kind* (deconvolution) Volterra problem and is ill-posed. The book never inverts in that direction for estimation; it always poses a well-posed second-kind forward solve inside a likelihood.

Appendix B

Transforms, ODEs, and state space

The methods of this appendix apply to the convolution case $K(t, u) = \phi(t - u)$; their failure off it is what makes excitation covariates hard (Ch. 10).

B.1 Laplace, Fourier, and the convolution theorem

The transforms turn convolution into multiplication, $\mathcal{L}\{\phi * \xi\} = \mathcal{L}\{\phi\}\mathcal{L}\{\xi\}$. Applying $\mathcal{L}$ to $\xi = s + \phi * \xi$,

$$\mathcal{L}\{\xi\} = \frac{\mathcal{L}\{s\}}{1 - \mathcal{L}\{\phi\}}, \quad \hat{\xi}(\omega) = R(\omega)\hat{s}(\omega), \quad R = \frac{1}{1 - \hat{\phi}}. \quad (\text{B.1})$$

R is the *transfer function* (the resolvent of Appendix A in the frequency domain). For the exponential kernel $\hat{\phi} = \kappa\theta/(\theta + i\omega)$, so $R(\omega) = (\theta + i\omega)/((1 - \kappa)\theta + i\omega)$ — a one-pole filter whose pole is the ODE rate below.

Intuition

Transforms re-express ξ in pure oscillations; a shift-invariant system scales each independently, so the tangled convolution becomes a per-frequency multiply. A covariate that changes the gain over time breaks shift-invariance — hence no single transfer function in Ch. 10, and one must return to the time domain.

B.2 From exponential kernel to a first-order ODE

Differentiating $y = \phi * \xi$ for $\phi = \kappa\theta e^{-\theta t}$ gives

$$y'(t) = (\kappa - 1)\theta y(t) + \kappa\theta s(t), \quad \xi = s + y, \quad (\text{B.2})$$

a stable first-order ODE forced by the baseline (pole $(\kappa - 1)\theta < 0$).

Theorem B.1 (realisation). *The convolution MBPP equation is equivalent to a finite linear constant-coefficient ODE iff ϕ has a strictly proper rational Laplace transform — equivalently, ϕ is a finite exponential-polynomial.*

A sum of Q exponentials gives the Q -state realisation $\dot{u} = Au + a s$, $\xi = s + \mathbf{1}^\top u$ (Ch. 11); the multivariate kernel gives the M^2 -state system of Ch. 12.

B.3 Time-varying systems and the fractional case

For excitation covariates the realisation is the same in form but with a time-varying gain (Ch. 10): a *linear time-varying* ODE, integrable step by step (exactly per-regime for piecewise-constant covariates) but with no closed matrix exponential when the $A(t)$ do not commute — the differential-equations echo of “no constant transfer function.” For a power-law kernel the transform is *not* rational, no finite ODE exists, and the resolvent is a Mittag–Leffler function $E_{\alpha,\alpha}$ (the dynamics are fractional); $E_{1,1} = \exp$ recovers the exponential case. In practice one approximates the power law by a sum of exponentials (restoring a finite ODE) or solves by Volterra quadrature.

Appendix C

Estimation theory

C.1 IC-LL as a Bregman divergence

A Bregman divergence for strictly convex φ is $B_\varphi(x, y) = \varphi(x) - \varphi(y) - \langle x - y, \nabla \varphi(y) \rangle \geq 0$.

Proposition C.1. *With $\varphi(x) = \sum_i (x_i \log x_i - x_i)$, $\mathcal{L}_{\text{IC}} = \text{KL}(\mathbf{C}, \Xi) + \text{const}$. The squared generator $\frac{1}{2}\|x\|^2$ instead gives the least-squares loss.*

Proof. The MBPP makes the bucket counts C_i independent Poisson with mean $\Xi_i = \int_{\tau_{i-1}}^{\tau_i} \xi$. Up to the θ -independent constant $\sum_i \log C_i!$, the negative log-likelihood is $\mathcal{L}_{\text{IC}} = \sum_i (\Xi_i - C_i \log \Xi_i)$. Take $\varphi(x) = \sum_i (x_i \log x_i - x_i)$, so $\nabla \varphi(y)_i = \log y_i$. Then

$$B_\varphi(\mathbf{C}, \Xi) = \sum_i (C_i \log C_i - C_i) - (\Xi_i \log \Xi_i - \Xi_i) - \log \Xi_i (C_i - \Xi_i) = \sum_i (\Xi_i - C_i \log \Xi_i) + \underbrace{\sum_i (C_i \log C_i - C_i)}_{\text{const in } \theta}.$$

The bracketed term depends only on the data, so $\mathcal{L}_{\text{IC}} = B_\varphi(\mathbf{C}, \Xi) + \text{const} = \text{KL}(\mathbf{C}, \Xi) + \text{const}$. Replacing φ by $\frac{1}{2}\|x\|^2$ gives $\nabla \varphi(y) = y$ and $B_\varphi = \frac{1}{2}\|\mathbf{C} - \Xi\|^2$, the SSE. $\square$

Theorem C.2 (Bregman $\leftrightarrow$ exponential family; Banerjee et al.). *For a regular exponential family with cumulant ψ and mean $\mu = \nabla \psi$, the negative log-likelihood equals $B_{\psi^*}(x, \mu)$ up to a constant. Poisson gives $\psi^*(\mu) = \mu \log \mu - \mu$ (so $B = \text{KL} - \text{IC-LL}$); Gaussian gives $\frac{1}{2}(x - \mu)^2$ (least squares).*

Choosing the divergence is choosing the noise model; KL is correct because a Poisson bucket has variance equal to its mean, so it weights bucket i by $1/\Xi_i$ (Ch. 6). The HIP method is least squares against the *intensity* under unit-bucket and discrete-convolution assumptions — a triply-restricted special case the MBPP framework derives and corrects.

C.2 Consistency, information, goodness of fit

Theorem C.3 (Ogata). *For a stationary, ergodic, subcritical process with θ_0 interior to a compact identifiable set and the usual smoothness/moment regularity, the MLE is consistent and asymptotically normal, $\sqrt{T}(\hat{\theta}_T - \theta_0) \rightarrow \mathcal{N}(0, \mathbf{i}(\theta_0)^{-1})$.*

SEs come from the observed information (Hessian of $-\ell$); under interval censoring, from $\mathcal{I}_{jk}^{\text{IC}} = \sum_i \partial_j \Xi_i \partial_k \Xi_i / \Xi_i$, inflated by $\sqrt{\text{dispersion}}$ for Hawkes over-dispersion (Ch. 7).

Theorem C.4 (time-rescaling). *If Λ is continuous and strictly increasing, the rescaled gaps $\Lambda(t_k) - \Lambda(t_{k-1})$ are i.i.d. $\text{Exp}(1)$.*

Hence the KS-against- $\text{Exp}(1)$ goodness-of-fit test (`gof.py`).

C.3 The κ - θ ridge, derived

We can see the ridge directly in the Fisher information. Write the interval-censored information $\mathcal{I}_{jk} = \sum_i \Xi_i^{-1} (\partial_j \Xi_i) (\partial_k \Xi_i)$ (the Poisson/KL weight $1/\Xi_i$ from the proposition above). The sensitivities $\partial_j \Xi_i$ flow from $\xi = s + \phi * \xi$ through the kernel parameters.

Proposition C.5 (near-degeneracy). *For $\phi = \alpha e^{-\beta t}$ with $\alpha = \kappa\theta$, $\beta = \theta$, the information $\mathcal{I}(\kappa, \theta)$ has a large eigenvalue with eigenvector $\approx \partial_\kappa$ and a small eigenvalue with eigenvector $\approx \partial_\theta$, and the small eigenvalue $\rightarrow 0$ as the bucket width $\Delta \rightarrow \infty$.*

Sketch. The total kernel mass is $\int_0^\infty \phi = \alpha/\beta = \kappa$, independent of θ . Differentiating the resolvent identity $\xi = s + \phi * \xi$ gives, in the stationary/constant-baseline regime, $\partial_\kappa \xi = \partial_\kappa(\text{mass}) \cdot (\text{gain})$ with gain $1/(1-\kappa)$, a θ -free first-order shift of the *level* of ξ . By contrast $\partial_\theta \phi = \alpha e^{-\beta t} (1-\beta t)/\theta \cdot (\dots)$ has zero integral ($\int_0^\infty \partial_\theta \phi = \partial_\theta(\alpha/\beta) = 0$ along constant κ): changing θ *reshapes* ϕ in time without changing its mass, so it perturbs only the *timing* of ξ , not its level. A censoring bucket reports $\Xi_i = \int_{\tau_{i-1}}^{\tau_i} \xi$, which averages over exactly that timing; as $\Delta \rightarrow \infty$ the bucket integral becomes insensitive to the shape perturbation, $\partial_\theta \Xi_i \rightarrow 0$, and the corresponding eigenvalue of $\mathcal{I}$ vanishes while $\partial_\kappa \Xi_i$ (a level shift) persists. Hence κ stays identified, θ degrades, and the weak eigenvector aligns with constant- κ curves. $\square$

Remark. This is why the Bayesian posterior of Chapter 8 is a banana along constant- κ curves, why the Laplace covariance $\mathcal{I}^{-1}$ blows up in the θ direction, and why an informative prior on θ (not more buckets) is the correct remedy. Excitation covariates add non-convexity and require covariate–event co-occurrence to identify δ .

Practical guidance: trust κ and covariate effects γ ; fix or prior-constrain θ ; report wide intervals for it.

C.4 Bernstein–von Mises: from likelihood to posterior

The Bayesian and frequentist large-sample stories coincide. Let $\ell_n(\theta) = -\mathcal{L}_{\text{IC}}(\theta)$ be the log-likelihood from n buckets and $\hat{\theta}_n$ the MLE.

Theorem C.6 (LAN $\Rightarrow$ BvM). *Suppose ℓ_n is locally asymptotically normal at θ_0 : $\ell_n(\theta_0 + h/\sqrt{n}) - \ell_n(\theta_0) = h^\top \Delta_n - \frac{1}{2} h^\top \mathcal{I} h + o_P(1)$ with $\Delta_n \rightarrow \mathcal{N}(0, \mathcal{I})$ and $\mathcal{I} \succ 0$, the prior π is continuous and positive at θ_0 , and the MLE is consistent. Then the posterior for $\vartheta = \sqrt{n}(\theta - \hat{\theta}_n)$ converges in total variation to $\mathcal{N}(0, \mathcal{I}^{-1})$.*

Sketch. Write the posterior density of ϑ as $p_n(\vartheta) \propto \exp\{\ell_n(\hat{\theta}_n + \vartheta/\sqrt{n}) - \ell_n(\hat{\theta}_n)\} \pi(\hat{\theta}_n + \vartheta/\sqrt{n})$. A second-order expansion at the MLE (where $\nabla \ell_n = 0$) gives the exponent $-\frac{1}{2} \vartheta^\top \mathcal{I} \vartheta + o_P(1)$ uniformly on compacts, and the prior factor $\rightarrow \pi(\theta_0)$ by continuity. Thus $p_n(\vartheta) \rightarrow c e^{-\frac{1}{2} \vartheta^\top \mathcal{I} \vartheta}$ pointwise; Scheffé’s lemma upgrades this to total-variation convergence to $\mathcal{N}(0, \mathcal{I}^{-1})$. The prior washes out at rate $1/\sqrt{n}$ — but *only* in directions where $\mathcal{I} \succ 0$. $\square$

Remark. Proposition C.5 is exactly the failure mode: along the θ eigenvector $\mathcal{I} \rightarrow 0$, the $o_P(1)$ remainder is not negligible relative to the (vanishing) curvature, and the posterior does *not* forget the prior there. BvM holds on the identified subspace (κ, γ) and fails on the ridge — precisely matching the wide θ credible intervals we observe.

Appendix D

API cheat-sheet

A task-oriented index into the `hawkes_calibration` package. Import names are top-level (`from hawkes_calibration import ...`) unless a submodule is given.

D.1 “I want to ...”

Task	Use
simulate a Hawkes process (event times)	<code>simulate_multivariate_hawkes</code>
simulate keeping immigrants/offspring separate	<code>simulate_separable_hawkes</code>
simulate with covariate-modulated excitation	<code>simulate_hawkes_excitation</code>
turn event times into bucket counts	<code>interval_censor</code> , <code>uniform_obs.times</code>
build an MBPP, get ξ / Ξ	<code>MBPP(ExponentialKernel(...), exo).intensity/compensator</code>
solve for an arbitrary forcing / sum-of-exp	<code>operators.solve_mbpp_ode</code>
solve the multivariate MBPP	<code>operators.solve_mbpp_ode_multivariate</code>
solve with excitation covariates (LTV)	<code>operators.solve_mbpp_ltv</code>
solve a general (power-law) Volterra MBPP	<code>operators.solve_mbpp_volterra</code>
fit (κ, θ) from counts	<code>fit_mbpp_ic(loss="ic-ll")</code>
fit jointly over many sequences	<code>fit_mbpp_ic_multi</code>
fit baseline covariates γ	<code>fit_mbpp_ic_covariates</code>
fit excitation covariates δ	<code>fit_mbpp_ic_excitation</code>
fit a multi-timescale kernel	<code>fit_mbpp_ic_sumexp</code>
forecast future counts	<code>forecast_counts</code>
get standard errors	<code>ICFitResult.se_kappa/se_theta</code> (quasi-Poisson)
check goodness of fit	<code>gof:</code> <code>time_rescaling_residuals</code> , <code>ks_test_exp1</code> , <code>dispersion</code>
learn the operator / kernel	<code>SpectralOperator</code> , <code>operators_nn.DeepONetOperator</code>
amortise inference (counts→params)	<code>operators_nn.AmortizedInference</code>
high-dim TF operators	<code>operators_tf</code> (FNO, DeepONet, SSM)
neural PDE solver (PINN / PINO)	<code>neural_solver.make_neural_solver(backend, mode)</code>
exact anchors / training diagnostics	<code>neural_solver.make_anchor_data</code> , <code>solver_accuracy_report</code>

D.2 Parameter conventions

The theory uses $\phi(t) = \kappa\theta e^{-\theta t}$ (κ = branching ratio, θ = decay); the event-time code uses $\phi(t) = \alpha e^{-\beta t}$. They match via $\alpha = \kappa\theta$, $\beta = \theta$, i.e. $\kappa = \alpha/\beta$; convert with `kappa.theta_to.alpha_beta`

/ `alpha_beta_to_kappa_theta`.

D.3 Further reading in the repository

the package `README` (the practical reference: a status matrix of every model/solver/fitter, the learned-solver and PINN/PINO recipe with hybrid training, and a noise-stress harness for the TF models), and the `tests/` suite (the executable ground truth for every numerical claim in this book).

Bibliography

- [1] A. G. Hawkes (1971). Spectra of self-exciting and mutually exciting point processes. *Biometrika* 58(1).
- [2] A. G. Hawkes, D. Oakes (1974). A cluster process representation of a self-exciting process. *J. Appl. Probab.* 11(3).
- [3] A. Dassios, H. Zhao (2013). Exact simulation of Hawkes process with exponentially decaying intensity. *Electron. Commun. Probab.* 18.
- [4] M.-A. Rizoïu et al. (2017). Expecting to be HIP: Hawkes intensity processes for social media popularity. *WWW*.
- [5] A. Banerjee, S. Merugu, I. Dhillon, J. Ghosh (2005). Clustering with Bregman divergences. *JMLR* 6.
- [6] Y. Ogata (1978). Asymptotic behaviour of MLEs for stationary point processes. *Ann. Inst. Statist. Math.* 30.
- [7] M.-A. Rizoïu, A. Soen, S. Li, P. Calderon, L. J. Dong, A. K. Menon, L. Xie (2022). Interval-censored Hawkes processes. *JMLR* 23(1):1–84.
- [8] G. Gripenberg, S.-O. Londen, O. Staffans (1990). *Volterra Integral and Functional Equations*. Cambridge.
- [9] T. Chen, H. Chen (1995). Universal approximation to nonlinear operators by neural networks. *IEEE TNN* 6(4).
- [10] Z. Li et al. (2021). Physics-Informed Neural Operator for learning PDEs. *arXiv:2111.03794*.
- [11] H. Haario, E. Saksman, J. Tamminen (2001). An adaptive Metropolis algorithm. *Bernoulli* 7(2).
- [12] A. Gelman, J. Carlin, H. Stern, D. Dunson, A. Vehtari, D. Rubin (2013). *Bayesian Data Analysis*, 3rd ed. CRC Press.
- [13] A. W. van der Vaart (1998). *Asymptotic Statistics*. Cambridge. (Bernstein–von Mises, Ch. 10.)